

Постановка задачи.

1. Представить ориентированный граф, состоящий из 7-10 вершин, с помощью матрицы смежности, а затем выполнить следующие операторы над его элементами.
2. Указать вершину v и определить список вершин, смежных с вершиной v . Если v не имеет смежных вершин, то возвращается «нулевая» вершина.
3. Указать вершину v и определить список вершин, из которых можно попасть в вершину v . Если таких вершин на орграфе нет, то возвращается «нулевая» вершина.
4. Определить кратчайшие пути от вершины-источника до всех вершин орграфа на основе алгоритма Дейкстры.

Листинг программы:

```
#define _CRT_SECURE_NO_WARNINGS
#include <iostream>
#include <iomanip>
#include <conio.h>
#include <vector>
#include <list>
#include <queue>
#include <string>
#include <stdio.h>
#include <stdlib.h>
using namespace std;

// Генерация случайного целого числа в диапазоне
int randInt(int min, int max) { return rand() % (max - min + 1) + min; }

// Валидация ввода целого числа
void validateInt(int& a, string varName)
{
    string sA = "";
    do
    {
        try
        {
            if (varName != "")
                cout << "Введите " << varName << ": ";
            cin >> sA;
            a = stoi(sA);
            break;
        }
        catch (invalid_argument ex) {
            cout << "Введенные данные не являются числом" << endl;
        }
        catch (out_of_range ex) {
            cout << "Введено слишком большое число" << endl;
        }
    } while (true);
}

// Вывод списка
void printVector(vector<int> v) {
    cout << "\n";
    for (int i = 0; i < v.size(); i++) {
        cout << v[i] << " ";
    }
}
```

```

// Вывод двумерного списка
void print2DVector(vector<vector<int>> v) {
    cout << "\n";
    for (int i = 0; i < v.size(); i++) {
        for (int j = 0; j < v[i].size(); j++) {
            cout << v[i][j] << " ";
        }
        cout << endl;
    }
}

// Граф
class Graph
{
    int numVertices; // кол-во вершин
    vector<vector<int>> adjLists; // список смежности

public:
    Graph(int V); // конструктор
    void addEdge(int src, int dest); // добавление вершины
    vector<int> findAdjacentVertexes(int v); // поиск смежных вершин
    void findAchievableVertexes(int v, vector<int>& result); // поиск достижимых
    // вершин
    bool isAdjVertex(int src, int dest); // проверка вершин на смежность
    int** createAdjacentMatrix(); // создание матрицы смежности
    vector<vector<int>> getAdjLists() { // получить список вершин
        return adjLists;
    }
};

// Конструктор графа
Graph::Graph(int vertices)
{
    numVertices = vertices;
    adjLists = vector<vector<int>>(vertices);
    for (int i = 0; i < adjLists.size(); i++) {
        adjLists[i] = vector<int>();
    }
}

// Добавление вершины
void Graph::addEdge(int src, int dest)
{
    for (int i = 0; i < adjLists[src].size(); i++) {
        if (adjLists[src][i] == dest)
            return;
    }
    adjLists[src].push_back(dest);
}

// Проверка вершин на смежность
bool Graph::isAdjVertex(int src, int dest)
{
    for (int j = 0; j < adjLists[src].size(); j++) {
        if (adjLists[src][j] == dest)
            return true;
    }
    return false;
}

// Поиск смежных вершин
vector<int> Graph::findAdjacentVertexes(int v)
{

```

```

    vector<int> vec;
    for (int i = 0; i < adjLists[v].size(); i++)
        vec.push_back(adjLists[v][i]);
    return vec;
}

// Поиск достижимых вершин
void Graph::findAchievableVertexes(int v, vector<int> &result)
{
    for (int i = 0; i < adjLists.size(); i++) {
        for (int j = 0; j < adjLists[i].size(); j++) {
            if (v == adjLists[i][j]) {
                bool isVertexVisited = false;
                for (int k = 0; k < result.size(); k++) {
                    if (i == result[k])
                        isVertexVisited = true;
                }
                if (!isVertexVisited) {
                    result.push_back(i);
                    findAchievableVertexes(i, result);
                }
            }
        }
    }
}

// Создание матрицы смежности
int** Graph::createAdjacentMatrix() {
    int** adjMatrix = new int* [numVertices];
    for (int i = 0; i < numVertices; i++)
        adjMatrix[i] = new int[numVertices];

    cout << "\n";
    for (int i = 0; i < numVertices; i++) {
        for (int j = 0; j < numVertices; j++) {
            adjMatrix[i][j] = isAdjVertex(i, j);
            cout << adjMatrix[i][j] << " ";
        }
        cout << endl;
    }

    return adjMatrix;
}

int main()
{
    system("chcp 1251");
    system("color 07");
    system("cls");
    Graph g(7);

    cout << "Выберите пункт меню\n1. Ввести граф вручную\n2. Заполнить случайными числами в диапазоне\n3. Использовать заготовленный граф\n";
    int menu;
    validateInt(menu, "");
    switch (menu)
    {
        case 1: // Ввести граф вручную
        {
            int a, n;
            cout << "Сколько вершин будет вводиться? [7, 10]\n";
            do
            {
                validateInt(n, "n");
            }
        }
    }
}

```

```

    } while (n < 7 || n > 10);
    g = Graph(n);

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++)
        {
            cout << "\nВершина: " << i + 1 << "\n";
            cout << "Есть ли дуга в вершину: " << j + 1 << "? (Y/N)";
            char c = _getch();
            if (c == 'Y' || c == 'y')
                g.addEdge(i, j);
        }
    }
}
break;
case 2: // Заполнить случайными числами в диапазоне
{
    g = Graph(10);
    int min = 0, max = 9;

    for (int i = 0; i < 10; i++) {
        g.addEdge(randInt(min, max), randInt(min, max));
    }
}
break;
default: // Использовать заготовленный граф

    g = Graph(7);

    g.addEdge(0, 1);
    g.addEdge(0, 2);
    g.addEdge(1, 2);
    g.addEdge(2, 3);
    g.addEdge(3, 0);

    g.addEdge(5, 4);
    g.addEdge(3, 5);
    g.addEdge(6, 5);

    break;
}
g.createAdjacentMatrix();
int v;

// 2. Указать вершину v и определить список вершин, смежных с вершиной
// v. Если v не имеет смежных вершин, то возвращается «нулевая» вершина.
cout << "2. Указать вершину v и определить список вершин, смежных с вершиной v.
Если v не имеет смежных вершин, то возвращается «нулевая» вершина.\n";
validateInt(v, "вершину");
vector<int> result = g.findAdjacentVertexes(--v);
for (int i = 0; i < result.size(); i++) {
    result[i] += 1;
}
printVector(result);
cout << "\n";

// 3. Указать вершину v и определить список вершин, из которых можно
// попасть в вершину v. Если таких вершин на орграфе нет, то возвращается
«нулевая» вершина.
cout << "3. Указать вершину v и определить список вершин, из которых можно
попасть в вершину v. Если таких вершин на орграфе нет, то возвращается «нулевая»
вершина.\n";
validateInt(v, "вершину");
result = { --v };

```

```

g.findAchievableVertexes(v, result);
for (int i = 0; i < result.size(); i++) {
    result[i] += 1;
}
printVector(result);
cout << "\n";

```

// 4. Определить кратчайшие пути от вершины-источника до всех вершин орграфа на основе алгоритма Дейкстры.

cout << "4. Определить кратчайшие пути от вершины-источника до всех вершин орграфа на основе алгоритма Дейкстры.\n";

```

{
    const int VERT_SIZE = 7;
    int a[VERT_SIZE][VERT_SIZE]; // матрица связей
    int d[VERT_SIZE]; // минимальное расстояние
    int v[VERT_SIZE]; // посещенные вершины
    int temp, minindex, min;
    int begin_index = 0;

    cout << "Алгоритм Дейкстры\nИнициализация матрицы связей\n";
    // Инициализация матрицы связей
    for (int i = 0; i < VERT_SIZE; i++) {
        a[i][i] = 0;
        for (int j = i + 1; j < VERT_SIZE; j++) {
            printf("Введите расстояние %d - %d: ", i + 1, j + 1);
            scanf("%d", &temp);
            a[i][j] = temp;
            a[j][i] = temp;
        }
    }

    // Вывод матрицы связей
    for (int i = 0; i < VERT_SIZE; i++) {
        for (int j = 0; j < VERT_SIZE; j++)
            printf("%5d ", a[i][j]);
        printf("\n");
    }

    // Инициализация вершин и расстояний
    for (int i = 0; i < VERT_SIZE; i++) {
        d[i] = 10000;
        v[i] = 1;
    }
    d[begin_index] = 0;

    // Шаг алгоритма
    do {
        minindex = 10000;
        min = 10000;
        for (int i = 0; i < VERT_SIZE; i++) {
            // Если вершину ещё не обошли и вес меньше min
            if ((v[i] == 1) && (d[i] < min)) {
                // Переписываем значения
                min = d[i];
                minindex = i;
            }
        }

        // Добавляем найденный минимальный вес к текущему весу вершины
        // и сравниваем с текущим минимальным весом вершины
        if (minindex != 10000) {
            for (int i = 0; i < VERT_SIZE; i++) {
                if (a[minindex][i] > 0) {
                    temp = min + a[minindex][i];

```

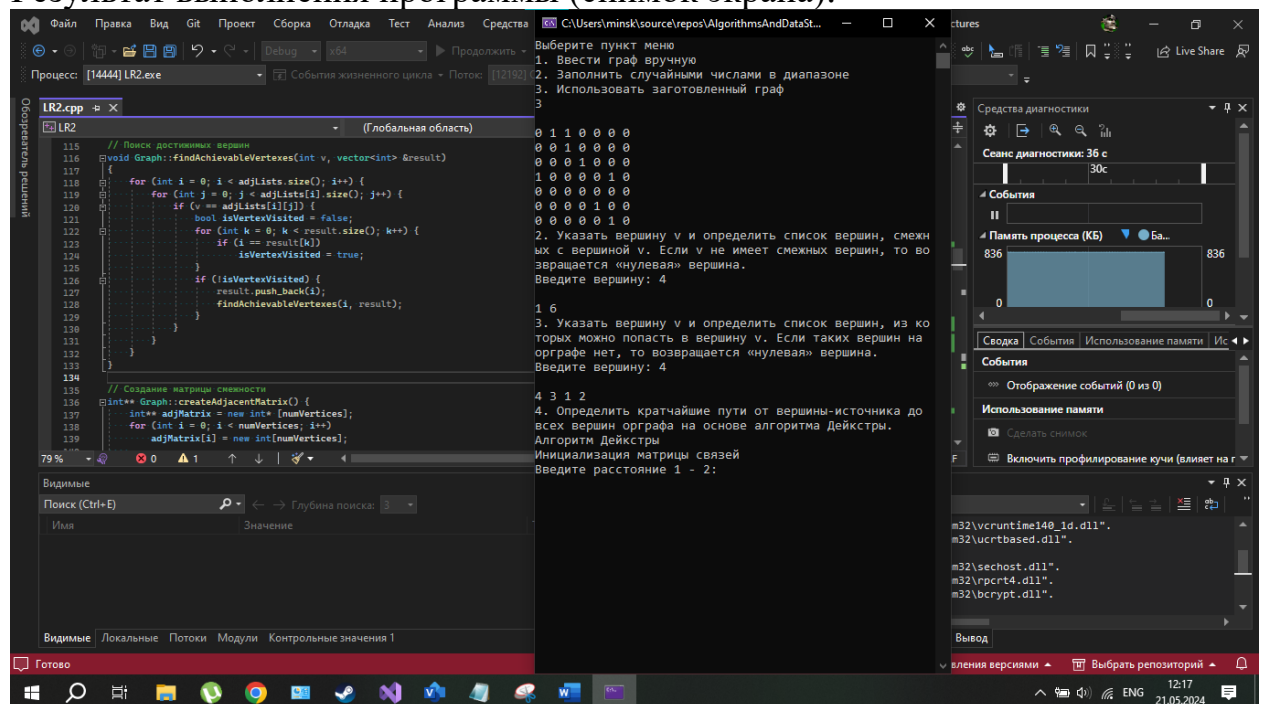
```

        if (temp < d[i]) {
            d[i] = temp;
        }
    }
    v[minindex] = 0;
}
} while (minindex < 10000);

// Вывод кратчайших расстояний до вершин
printf("\nКратчайшие расстояния до вершин: \n");
for (int i = 0; i < VERT_SIZE; i++)
    printf("%5d ", d[i]);
}
cout << "\n";
system("pause");
return 0;
}

```

Результат выполнения программы (снимок экрана):



Ответы на контрольные вопросы:

1. Из каких элементов состоит ориентированный граф?
Вершины и дуги.
2. Дать определение пути в ориентированном графе.
Путем на графе называется последовательность ребер, в которой конец одного ребра является началом следующего ребра.
3. Перечислите основные способы представления ориентированных графов.
Матрица смежности, список смежности, матрица инцидентности.
4. Для решения какой задачи на ориентированном графе удобно использовать алгоритм Дейкстры?
Нахождение кратчайшего пути с одним источником.
5. Для решения какой задачи на ориентированном графе удобно использовать алгоритм Флойда?
Нахождение путей между каждой парой вершин на орграфе.
6. Какая вершина графа называется его центром?
Центром орграфа G называется вершина с минимальным эксцентриситетом, т.е. это вершина, для которой максимальное расстояние (длина пути) до других вершин минимально.